

MediAssist AI

Intelligent Medical Triage & Diagnostics Dashboard

Technical Architecture & System Documentation

A comprehensive reference manual detailing features, databases, APIs, code logic, and execution parameters.

Prepared by:
Antigravity AI Coding Assistant

Date:
May 21, 2026

Version:
1.0.0 (Release)

Table of Contents

- 1. Executive Summary & Disclaimer..... Page 2
- 2. System Architecture & Directory Map..... Page 3
- 3. Frontend Technologies & Animations Page 4
- 4. AI Symptom Triage & Geolocation Systems Page 5
- 5. Medical Report Uploads & BMI Calculator..... Page 6
- 6. API Endpoints & Integration Schemas..... Page 7
- 7. Deployment, Orchestration & Troubleshooting Page 8

1. Executive Summary & Disclaimer

MediAssist AI is a premium full-stack medical helper designed to guide users through basic health assessments. Built as a dual-component architecture consisting of a React-Vite dashboard and an Express API server, the system harnesses Google Gemini AI for advanced medical analytics, report translation, and healthcare navigation.

By bridging the gap between complicated clinical report structures and patients, MediAssist AI decreases search fatigue and empowers individuals to take informed steps toward health resolution. It streamlines the initial pre-care stage, suggesting specific doctors and physical clinics based on real-time parameters.

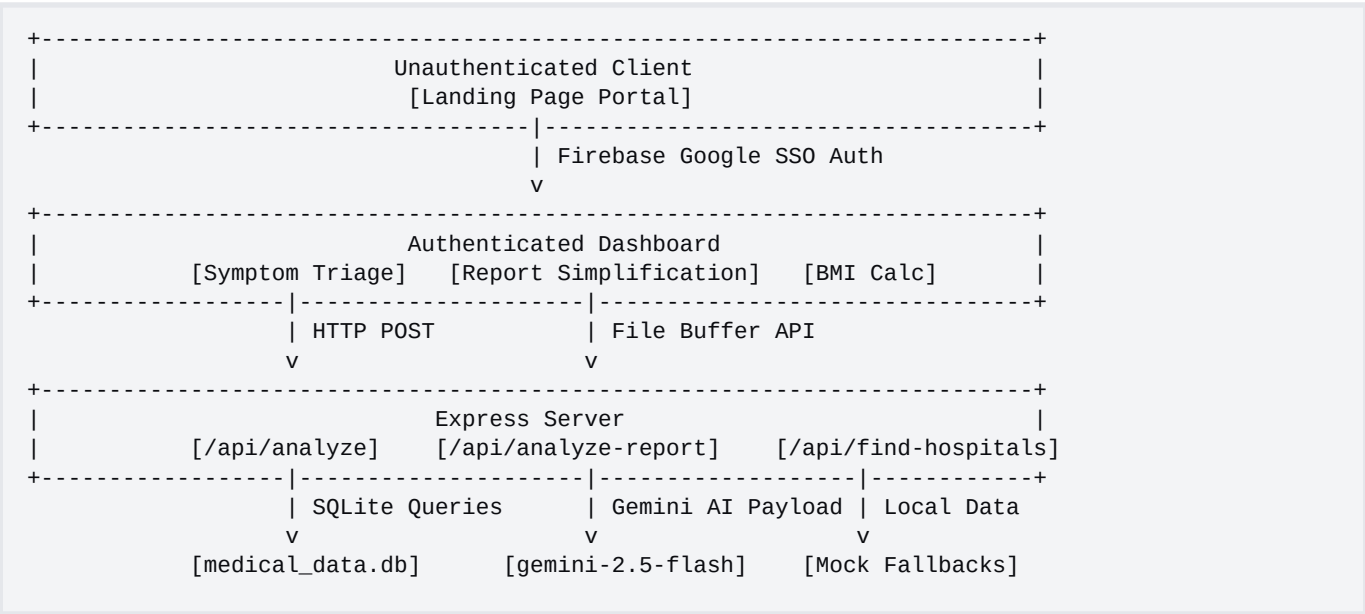
DISCLAIMER

MediAssist AI is not a registered medical diagnosis platform and does not possess professional medical certification. The system is designed to provide informative triage and recommendation references based on patient metrics. In emergency circumstances, contact local emergency services immediately.

2. System Architecture & Directory Map

The application executes in a clean Client-Server configuration. The React frontend interacts with the Express server via asynchronous HTTP requests. The backend leverages local SQLite databases for persistence and forwards complex natural language processing tasks to Google Generative AI.

System Architecture Flow



Project Workspace Directory Structure

The workspace is structured into active directories for the medical system alongside legacy files from a movie application. The key active locations are detailed below:

- `medical-dashboard/`: Renders the client-facing UI. This directory features the React modules, CSS system variables, and Firebase initialization scripts.
- `medical-backend/`: Serves the API routes, processes images, holds the SQLite schema, and acts as the gateway to the Gemini AI API endpoints.
- `medical-backend/server.js`: Orchestrates endpoints, manages Multer disk-storage, and registers error/rate-limiting fallback templates.
- `medical-dashboard/src/App.jsx`: Orchestrates state management for symptoms, BMI metrics, location triggers, dashboard view routing, and auth state tracking.

3. Frontend Technologies & Animations

The frontend UI emphasizes rich aesthetics, response-triggered micro-animations, and full compatibility. It avoids stock browser components, utilizing customized HSL color maps and border rules to construct a modern workspace.

Core Technologies

- Tailwind CSS: Implements grid layouts, color systems, and interactive styles (such as backdrop-blur adjustments).
- Lucide React: Supplies lightweight svg iconography.
- Firebase SDK: Maintains secure sessions using client credentials, reducing user login friction.

3D Parallax Landing Portal

The landing page is designed to captivate visitors through high-performance pseudo-3D scroll animations managed by Framer Motion. Using the browser's scrolling progress, the layout dynamically transforms visual properties:

- Hero section elements adjust their relative Y-axis and rotate backward (rotateX) as the user scrolls, creating a parallax depth effect.
- The main authentication container tilts forward and scales on cursor hover via spring physics.
- Cards in the "Why Choose Us" segment dynamically calculate rotation coefficients on mouse move to align with cursor direction.

Firebase Google SSO Integration

Authentication is fully automated. When a user clicks "Portal Login" or "Continue with Google", the Firebase SDK triggers a secure Google popup window. Once the login succeeds, the profile photo, email, and display name are bound to the client state. App.jsx registers a persistent auth observer:

```
useEffect(() => {
  const unsubscribe = onAuthStateChanged(auth, (currentUser) => {
    setUser(currentUser);
    setAuthLoading(false);
  });
  return () => unsubscribe();
}, []);
```

This listener guarantees that returning patients automatically bypass the auth screen and immediately access their personal health records upon opening the page.

4. AI Symptom Triage & Geolocation Systems

The diagnostic core of the application centers on the Symptom Triage view. It translates user-described symptoms into structured medical data.

Symptom Input & Tagging

The dashboard provides a dedicated symptom control board where patients register their age, gender, and append multiple symptom strings to a reactive tag array. Users also set a pain severity index from 1 (mild) to 10 (severe) using an styled slider.

Prompt Engineering & AI Evaluation

When the user submits their data, the backend server forwards the patient demographics to the Google Gemini model. It uses detailed developer instructions to enforce strict structure adherence. The prompt mandates that the AI return a structured JSON string matching a predefined format:

```
const prompt = `You are an expert AI Triage Assistant. Analyze this case:
Demographics: ${age} years old, ${gender}
Pain Level (0-10): ${painLevel}
Symptoms: ${symptoms.join(', ')}

Return response strictly in JSON matching this structure:
{
  "riskLevel": "LOW, MODERATE, or HIGH",
  "alert": "e.g., Immediate Attention Required",
  "analysis": "A detailed 2-paragraph explanatory assessment...",
  "emergencyAdvice": "Short emergency instructions...",
  "probableConditions": [
    { "name": "Condition 1", "percentage": 85, "color": "orange" }
  ],
  "dietaryAdvice": {
    "foodsToEat": ["List of optimal foods..."],
    "foodsToAvoid": ["List of foods to avoid..."]
  },
  "specialist": { "role": "Recommended Specialist Role" }
}`;
```

Hospital Locator & Geolocation

After receiving diagnostic results, the user can click "Find Nearby Hospitals". The application invokes the browser Geolocation API to fetch coordinates:

- If coordinates are granted, they are sent to the backend. If blocked, coordinates default to Hyderabad, India.
- The backend forwards coordinates and suspected conditions to Gemini, requesting three specialized physical hospitals nearby.
- The dashboard displays the address, specialties, and outputs a clickable link to Google Maps generated using standard URL parameters.

5. Medical Report Uploads & BMI Calculator

Beyond immediate symptom analysis, MediAssist AI assists with health document processing and vital tracking.

Multimodal Report Simplification

Dense medical jargon in diagnostic reports can cause patient anxiety. To solve this, the application features an AI Report Simplification tool:

- The user uploads a PDF or scan of a report from the sidebar.
- The file is captured by the Express backend using Multer disk-storage.
- The backend converts the file buffer to a base64 inline data payload and passes it to Gemini along with instructions to "read, simplify, and explain this report to a patient in everyday language."
- The simplified text is rendered inside a scrollable glass panel under the "Reports" tab, highlighting primary readings, irregularities, and doctor follow-up items.

Body Mass Index (BMI) Module

The dashboard provides a lightweight, fully functional BMI calculator. Users enter their height in centimeters and weight in kilograms. The frontend calculates the index and maps it to WHO standard classifications:

```
const heightInMeters = parseFloat(height) / 100;  
const bodyWeight = parseFloat(weight);  
const bmi = bodyWeight / (heightInMeters * heightInMeters);
```

Results are presented with visual feedback:

- Underweight (< 18.5): Styled in blue; suggests dietary consulting.
- Normal (18.5 - 24.9): Styled in emerald; congratulates healthy range.
- Overweight (25 - 29.9): Styled in yellow; suggests exercise and balanced diet.
- Obese (30+): Styled in red; advises consulting a medical practitioner.

A colored reference bar visually demonstrates the user's placement along the global BMI index scale.

6. API Endpoints & Integration Schemas

The backend service acts as a robust gateway for the React dashboard. It wraps the Google Generative AI API and provides local data backups.

Route: POST /api/analyze

Evaluates patient demographics, symptoms, and pain metrics.

Request Headers

```
Content-Type: application/json
```

Sample JSON Payload

```
{
  "symptoms": ["cough", "headache", "throat congestion"],
  "age": "32",
  "gender": "Male",
  "painLevel": 3
}
```

Route: POST /api/analyze-report

Uploads and interprets documents using multimodal machine learning.

Request Headers

```
Content-Type: multipart/form-data
```

Payload Format

```
Key: "report" (binary file attachment - PDF, JPG, PNG)
```

Route: POST /api/find-hospitals

Retrieves geo-mapped clinics specialized in suspected health conditions.

Sample JSON Payload

```
{
  "latitude": 17.3850,
  "longitude": 78.4867,
  "conditions": [
    { "name": "Acute Bronchitis", "percentage": 75 }
  ]
}
```

Robust Fallback System Architecture

To guarantee system stability, the Express router includes fallback mechanisms. If the Gemini API key is missing or blocked, the endpoints trigger structured fallback responses:

- The triage route falls back to a simulated Moderate Risk viral infection response.
- The report analyzer returns a mock report explanation reflecting stable vitals.
- The hospital locator outputs Apollo Hospitals (Jubilee Hills), KIMS (Secunderabad), and CARE Hospitals (Banjara Hills) as standard physical care listings in Hyderabad.

7. Deployment, Orchestration & Troubleshooting

Deploying and maintaining the application requires launching both the React frontend build pipeline and the Node.js Express server.

Execution Sequence

Execute the following steps to start the application. Do not run start_app.bat, as it is legacy configuration code.

Step 1: Set Up Backend Server

```
cd medical-backend
npm install
node server.js
```

Step 2: Set Up Frontend Dashboard

On Windows PowerShell, use the CMD shell wrapper to start Vite to prevent script restriction blocks:

```
cd medical-dashboard
npm install
cmd /c "npm run dev"
```

Diagnostics & Troubleshooting Matrix

Issue	Probable Cause	Resolution
ERR_CONNECTION_REFUSED	Backend server is inactive or occupied.	Restart Node using node server.js on port 3000.
PowerShell script block	Windows execution policy restriction.	Start Vite server via cmd /c "npm run dev".
AI fallback triggering	Gemini API limit or missing API key.	Verify API key definition in server.js line 13.
Geolocation failure	Browser location access denied by user.	Defaults to Hyderabad. Allow location access in browser.
Empty Dashboard State	User dashboard accessed without auth.	Confirm Firebase config keys; log in via

Google SSO.

WARNING

Ensure the SQLite database (medical_data.db) is preserved within the medical-backend directory. If the database file is deleted or corrupted, execute "node seed.js" to reinitialize templates.

